# 深度强化学习在无人机飞行控制（冰湖环境）中的应用与研究

组员：朱嘉语 23354197 林妍汐 23354105 毛星懿 23354124 刘仲晗 23354117 刘礼豪 23354014

上午第 12 组

2026 年 1 月 2 日

## 摘要

针对复杂环境下无人机自主控制的实时性与鲁棒性需求，本文开展了从深度强化学习算法研究、分布式仿真验证到嵌入式实机部署的全流程开发工作。首先，通过构建高保真物理模型，在确定性与随机性环境下对比分析了多种强化学习算法的性能表现，揭示了算法在处理环境不确定性时的鲁棒性边界。其次，为解决大规模实验的数据管理难题，本项目构建了基于工厂模式与四级目录结构的高效仿真平台，实现了 800 余组并行实验的自动化调度与结果复现。在工程化落地方面，本文详细记录了算法从仿真环境向底层嵌入式平台的迁移过程，重点解决了串口通信协议优化、指令同步及信号断联等底层技术瓶颈。实验结果表明，尽管物理环境中的硬件约束与噪声干扰导致算法性能较仿真阶段有所回落，但通过系统性的参数调优，控制系统仍具备良好的稳定性。本项目不仅验证了算法的有效性，更通过完整的“开发-仿真-测试”流程，探索了复杂控制系统工程化实现的可靠路径。

**关键词：**深度强化学习；分布式仿真；实机部署；工程化实现；鲁棒性分析

## 1 实验目的与任务

### 1.1 实验目的

本实验旨在构建一个可扩展的强化学习实验平台，将无人机在复杂环境中的飞行控制问题抽象为经典 FrozenLake 环境，实现从理论建模、算法仿真到实机验证的全流程闭环。具体实验目的如下：

1. **掌握马尔可夫决策过程在无人机导航中的建模方法**

   将无人机飞行任务形式化为状态空间、动作空间与奖励函数的数学表达。

   通过 `environments/frozenlake_env.py` 实现可配置的环境交互接口，支持确定性与随机性两种物理模式，分别模拟理想控制与风力扰动场景。

2. **系统对比不同强化学习算法在路径规划中的性能表现**

   实现四大类共 16 种算法，涵盖：

   (a) 规划算法（`value_iteration.py`、`policy_iteration.py`、`astar.py`）

   (b) 表格型算法（`q_learning.py`、`sarsa.py`、`double_q_learning.py` 等）

   (c) 模型基础算法（`dyna_q.py`、`prioritized_sweeping.py`）

   (d) 深度强化学习算法（`dqn.py`、`double_dqn.py`、`dueling_dqn.py`、`per_dqn.py`）

   通过系统化的实验配置（`experiments/configs.py`）与运行框架（`experiment_runner.py`），量化对比各算法在不同地图规模与随机性模式下的性能差异。

3. **探究环境随机性对算法鲁棒性的影响机制**

   通过 `is_slippery` 参数控制环境状态转移的随机性，模拟无人机在风扰等现实扰动下的飞行表现。

   分析算法在确定性环境（理想控制）与随机性环境（现实扰动）下的收敛性、稳定性与路径安全性差异。

4. **构建自动化、可复现的大规模实验评估体系**

   开发一键式实验脚本（如 `run_full_benchmark.ps1`、`run_quick_benchmark.ps1`），支持批量运行、结果保存与可视化生成。

   建立结构化结果存储体系（`results/` 目录），按地图、模式、算法、种子四级目录组织实验数据，便于后续分析与复现。

5. **实现从仿真到实机的策略迁移与验证**

   将训练成熟的策略部署至真实无人机，通过飞控系统实现动作指令的实时下发与状态反馈。实际场地中分别进行确定性与随机性模式下的实机演示，验证实际物理环境中的可行性与鲁棒性，通过实时监控系统展示无人机位置与轨迹。

## 1.2 实验任务描述

### 1.2.1 基础任务

1. **环境配置与基础地图验证**

   基于 `environments/frozenlake_env.py` 构建标准 4*4 FrozenLake 环境。验证在 Deterministic（`is_slippery=False`）& Stochastic（`is_slippery=True`）两种模式下的基础交互功能。

2. **Q-Learning 算法实现与调参**

   在 `algorithms/tabular/q_learning.py` 中实现经典的 Q-Learning 算法。

   调整学习率、折扣因子、探索率等超参数，观察对收敛速度与最终策略的影响。

3. **基础模式对比分析**

   在 `4x4` 地图上，分别运行 Deterministic 与 Stochastic 模式下的 Q-Learning，记录成功路径、步数与收敛曲线。

### 1.2.2 进阶扩展任务

本组分别进行了系统性、工程化、可视化三方面的深度扩展：

1. **多尺度复杂地图体系构建**

   在 `environments/maps_icegrid.py` 中设计了 24 张结构化地图，涵盖：

   - 标准地图：`4x4_default`、`8x8_official`
   - 难度分级地图：从 `5x5_easy_*` 到 `10x10_hard_mixed`
   - 支持自定义地图描述符（`desc` 参数），实现障碍物分布的灵活配置。

2. **全系列算法库实现与系统集成**

   (a) 表格型进阶：实现 SARSA、Expected SARSA、N-step TD、Monte Carlo Control、Eligibility Traces 等，代码位于 `algorithms/tabular/`。

   (b) 深度强化学习：基于 PyTorch 实现 DQN、Double DQN、Dueling DQN、PER-DQN，代码位于 `algorithms/deep_value/`，支持神经网络拟合高维状态价值函数。

   (c) 规划算法作为基准：实现 A* 搜索、值迭代、策略迭代，作为" 上帝视角" 的最优路径基准，代码位于 `algorithms/planning/`。

   (d) 模型-based 算法：实现 Dyna-Q 与 Prioritized Sweeping，结合模型学习与规划，代码位于 `algorithms/model_based/`。

3. **自动化实验管理与大规模基准测试**

   开发 `experiments/configs.py` 定义五种实验预设，从 `quick_test` 到 `full_benchmark`，支持 800 个实验组合（16 算法 *5 地图 *2 模式 *5 种子）。

   通过 `run_experiments.py` 与 PowerShell 脚本实现实验的批量提交、进度监控与错误恢复。

4. **多维度可视化系统开发**

   在 `visualization/` 目录下构建完整的可视化工具包：

   (a) 静态图表生成（`static_plots.py`）：训练曲线、Q 值热力图、策略可视化。

   (b) 动态动画生成（`animation.py`）：策略执行动画、学习进度动画、实验对比动画。

   (c) 实验可视化集成（`generate_visualizations.py`）：一键生成所有图表与动画。

5. **结构化结果存储与复现保障**

   实验结果按地图/模式/算法/种子四级目录存储于 `results/`，每个实验包含：

   - `results.json`（完整配置与指标）
   - `training_history.json`（训练过程数据，用于动画回放）
   - `policy.npy/png`、`value_function.npy/png`、`training_curve.png` 等可视化文件。

   此外，我们提供 `requirements.txt` 完整依赖列表。编写 `README.md`、`README_BENCHMARK.md` 等文档，里面详细说明了实验配置、运行方法与故障排查。

# 2 强化学习算法原理 (Algorithm Introduction)

## 2.1 理论基础：马尔可夫决策过程 (MDP)

所有强化学习算法均在马尔可夫决策过程框架下运行，定义为五元组 $(S, A, P, R, \gamma)$：

- $S$：状态空间，表示无人机在网格中的位置。状态空间大小为 $n \times m$。
- $A$：动作空间，包括上、下、左、右四个方向。
- $P$：状态转移概率。在 Stochastic 模式下，实际执行动作与预期动作存在偏差，概率分布为：$P(s'|s,a) = 0.33$(预期方向) + 0.33 (相邻方向)。
- $R$：奖励函数。到达终点 $G$ 获得 $+1$，掉入坑洞 $H$/平地移动 $F$ 获得 0。
- $\gamma$：折扣因子，平衡当前奖励与远期利益，通常取值在 $[0, 1]$ 范围内。

## 2.2 规划类算法 (Planning Algorithms)

此类算法假设环境模型（$P$ 和 $R$）已知，通过动态规划求解最优策略，提供理论最优基准。

### 2.2.1 值迭代 (Value Iteration)

通过迭代求解贝尔曼最优方程直接更新状态值函数，直到收敛后提取贪心策略。(动态规划)，在 $4 \times 4$ 地图上通常需要 $10 - 20$ 次迭代即可收敛到最优解。值迭代在有限 MDP 中保证收敛到唯一最优解，收敛速度为几何级数 $O(\gamma^k)$。

### 2.2.2 策略迭代 (Policy Iteration)

交替执行策略评估（计算当前策略 $\pi$ 下的值函数 $V_\pi$）和策略改进（基于 $V_\pi$ 贪心更新策略两个步骤）。相比值迭代，策略迭代在相同精度要求下通常需要更少的迭代次数，但每次迭代的计算复杂度较高。策略迭代在有限 MDP 中保证在有限步内收敛到最优策略，通常比值迭代更快但每步计算量更大。

### 2.2.3 A* 启发式搜索

A* 搜索结合了 Dijkstra 算法的最优性和启发式搜索的效率，通过代价函数 $f(n) = g(n) + h(n)$ 指导搜索。采用曼哈顿距离作为启发函数以保证可采纳性，使用堆维护待探索节点，通过 `came_from` 字典记录搜索路径，当启发函数 $h(n)$ 满足可采纳性（不高于真实代价）且一致时，A* 保证找到最优路径。

## 2.3 表格型强化学习算法 (Tabular Reinforcement Learning)

当环境模型未知时，Agent 通过与环境交互获取经验进行学习。表格型算法使用表格数据结构存储状态-动作价值函数，适用于离散状态空间问题。

### 2.3.1 经典时序差分算法

- **Q-Learning 算法**：典型的异策略 (off-policy) 时序差分学习算法，通过最大化操作学习最优动作值函数。行动策略使用 $\epsilon$-greedy，目标策略为贪婪策略，每个时间步即时更新 Q 值。在满足适当条件下，Q-Learning 以概率 1 收敛到最优 Q 函数。

  核心更新公式：

  $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Delta Q$$

  其中：

  $$\Delta Q = r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)$$

- **SARSA 算法**：同策略 (on-policy) 算法，使用当前策略选择的下一动作更新 Q 值。与 Q-Learning 不同的是，该算法使用实际执行的下一动作 $a_{t+1}$ 而非最优动作，在随机环境中更保守，避免过于乐观的估计，通常比 Q-Learning 收敛慢但更稳定。

- **Expected SARSA 算法**：SARSA 的改进，使用下一状态的期望值而非采样值。相比 SARSA 减少因随机采样带来的方差，更新目标确定性更高，训练更稳定。

  核心更新公式：

  $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Delta Q_E$$

  其中：

  $$\Delta Q_E = r_{t+1} + \gamma \mathbb{E}_\pi[Q(s_{t+1}, a_{t+1})] - Q(s_t, a_t)$$

  期望值计算：

  $$\mathbb{E}_\pi[Q(s_{t+1}, a)] = \sum_a \pi(a|s_{t+1}) Q(s_{t+1}, a)$$

  其中 $\pi(a|s_{t+1})$ 是 $\epsilon$-greedy 策略的概率分布。

#### 2.3.2 高级表格型算法

- **Double Q-Learning 算法**：通过维护两个独立的 Q 表缓解 Q-Learning 的过估计 (overestimation) 问题。随机选择一个 Q 表进行更新，使用另一个 Q 表选择动作，$Q_1$ 和 $Q_2$ 独立学习。

  更新公式：

  $$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha \Delta Q_1$$

  其中：

  $$\Delta Q_1 = r_{t+1} + \gamma Q_2(s_{t+1}, a^*) - Q_1(s_t, a_t)$$

  且：

  $$a^* = \arg\max_a Q_1(s_{t+1}, a)$$

  过估计缓解原理：标准 Q-Learning 的过估计上界为 $\frac{\gamma}{1-\gamma}\epsilon$，其中 $\epsilon$ 是估计误差。Double Q-Learning 通过交叉评估显著降低这一偏差。

- **n 步时序差分学习**：平衡蒙特卡洛方法（完整回合）和一步 TD 学习，通过多步回报减小偏差。

  $n$ 步回报定义：

  $$G_t^{(n)} = \sum_{k=1}^{n} \gamma^{k-1} r_{t+k} + \gamma^n V(s_{t+n})$$

  对于 SARSA 形式：

  $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ G_t^{(n)} - Q(s_t, a_t) \right]$$

- **资格迹算法 (SARSA($\lambda$)/$Q(\lambda)$)** $TD$

  - 迹衰减：$E_t(s,a) = \gamma\lambda E_{t-1}(s,a) + \mathbb{I}\{s = s_t, a = a_t\}$
  - TD 误差：$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$
  - 全局更新：$Q(s,a) \leftarrow Q(s,a) + \alpha\delta_t E_t(s,a)$

  算法变体：

  - SARSA($\lambda$)
  - Q($\lambda$) $Watkins$

  参数 $\lambda$ 的影响：

  - $\lambda = 0$：退化为一步 TD
  - $\lambda = 1$：退化为蒙特卡洛方法
  - $0 < \lambda < 1$：平滑权衡，最近经验权重更高

#### 2.3.3 蒙特卡洛控制算法

基于完整回合的回报进行策略评估和改进。

算法流程：

1. 回合生成：使用当前策略与环境交互至终止
2. 回报计算：$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_{k+1}$
3. 首次访问/每次访问：更新策略
4. 策略改进：基于更新的 Q 值改进策略

更新公式（首次访问蒙特卡洛）：

$$Q(s,a) \leftarrow Q(s,a) + \alpha(G_t - Q(s,a))$$

其中 $G_t$ 是首次访问 $(s,a)$ 后的累积折扣回报。

异策略蒙特卡洛（通过重要性采样实现）：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\rho_t(G_t - Q(s,a))$$

其中：

$$\rho_t = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}$$

是重要性采样比率。

#### 2.3.4 收敛性理论分析

- **Robbins-Monro 条件**：表格型算法收敛的充分条件：

  $$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

- **GLIE 条件（贪心极限下的无限探索）**：$\epsilon$-greedy 策略满足 GLIE 条件：

  $$\lim_{t\to\infty} N_t(s,a) = \infty \quad 和 \quad \lim_{t\to\infty} \epsilon_t = 0$$

  其中 $N_t(s,a)$ 是 $(s,a)$ 的访问次数。

### 2.4 深度强化学习 (Deep Reinforcement Learning)

传统表格型强化学习在状态空间维度爆炸时面临"维数灾难"，深度强化学习通过神经网络参数化价值函数，实现了对高维状态空间的泛化能力。

1. **DQN (Deep Q-Network)**：通过深度神经网络近似 Q 函数，并引入关键技术改进：

   - 经验回放缓冲区 (Experience Replay Buffer)：用于存储交互经验元组，打破时序相关性，提高样本效率。

- 目标网络 (Target Network)：固定目标 Q 值的计算网络，减少" 追逐移动目标" 问题，采用软更新（周期性地从策略网络复制参数）。

训练损失函数：

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(y - Q(s,a;\theta)\right)^2\right]$$

其中：$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$

2. **Double DQN**：针对 DQN 的过估计偏差问题，通过解耦动作选择与价值评估来改进。

过估计问题根源：在标准 DQN 中，目标值计算使用同一个网络进行最大化和评估，导致估计偏差的上界为 $\frac{\gamma}{1-\gamma}\epsilon$。

Double DQN 解决方案：

$$y_j^{\text{DDQN}} = r_j + \gamma Q\left(s_{j+1}, a^*; \theta^-\right)$$

其中：

$$a^* = \arg\max_{a'} Q(s_{j+1}, a'; \theta)$$

3. **Dueling DQN**：通过网络结构创新，将状态价值与动作优势分离，提升对状态价值的评估质量。

网络架构：

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|A|}\sum_{a'} A(s,a')$$

- 共享特征提取层：提取状态的高级特征
- 价值流 (Value Stream)：估计状态 $s$ 的长期价值 $V(s)$
- 优势流 (Advantage Stream)：评估每个动作 $a$ 相对于平均动作的优势 $A(s,a)$

4. **PER-DQN (Prioritized Experience Replay DQN)**：通过非均匀采样优化经验回放过程，优先回放高学习价值的经验。

优先级计算：

- TD 误差作为优先级：$p_i = |\delta_i| + \epsilon$
- 采样概率：$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$
- 重要性采样权重：$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta$

**深度强化学习理论局限性**

尽管深度强化学习在实践上取得成功，但仍存在理论挑战：

- 函数近似误差：神经网络作为通用函数逼近器，但可能引入近似误差
- 探索-利用权衡：$\epsilon$-贪心策略在连续动作空间中效率低
- 样本效率：相比监督学习，RL 的样本效率仍然较低
- 稳定性问题：训练过程可能不稳定，需要精心调参
- 可解释性：深度神经网络的决策过程缺乏可解释性

## 2.5 算法综合对比

### 2.5.1 算法综合对比（一）：规划与表格型算法

| 算法类别 | 算法名称 | 策略类型 | 更新方式 | 收敛性 |
|---|---|---|---|---|
| 3* 规划算法 | 值迭代 | 动态规划 | bellman | 理论最优 |
| | 策略迭代 | 动态规划 | 策略评估与改进 | 理论最优 |
| | A* 搜索 | 启发式搜索 | 代价函数最小化 | 最优路径 |
| 7* 表格型 RL | Q-Learning | 异策略 | 一步引导 | 渐进最优 |
| | SARSA | 同策略 | 一步引导 | 收敛到安全策略 |
| | Expected SARSA | 同策略 | 期望引导 | 更稳定收敛 |
| | DQ-Learning | 异策略 | 交叉引导 | 缓解过估计 |
| | n-step TD | 同/异策略 | n 步引导 | 权衡点最优 |
| | Eligibility Traces | 同/异策略 | 资格迹引导 | 高效传播信用 |
| | Monte Carlo | 同/异策略 | 完整回合 | 无偏但高方差 |

表 1: 规划与表格型强化学习算法对比

### 2.5.2 算法综合对比（二）：深度与模型基础算法

| 类别 | 算法 | 策略类型 | 更新方式 | 收敛性 |
|---|---|---|---|---|
| 4* 深度 RL | DQN | 异策略 | 神经网络近似 | 可能不稳定 |
| | Double DQN | 异策略 | 解耦选择评估 | 缓解过估计 |
| | Dueling DQN | 异策略 | 价值优势分离 | 更稳定 |
| | PER-DQN | 异策略 | 优先经验回放 | 效率高 |
| 2* 模型基础 | Dyna-Q | 异策略 | 模型学习 + 规划 | 效率高 |
| | Pri. Sweeping | 异策略 | 优先级更新 | 快速收敛 |

表 2: 深度与模型基础强化学习算法对比（Pri.：优先）

# 3 系统设计与实现 (System Implementation)

我们组采用模块化设计思想，将环境管理、算法实现、实验配置和结果可视化分离，确保系统的可扩展性、可维护性和可复现性。

## 3.1 环境构建系统

### 3.1.1 统一环境接口设计

基于 environments/frozenlake_env.py，我们实现了一个统一的 FrozenLake 环境包装器，为不同算法提供一致的交互接口。核心类结构支持自定义地图和随机性模式：

```
class FrozenLakeEnv:
    def __init__(self, map_name: str = "4x4_default",
                 is_slippery: bool = True,
                 render_mode: Optional[str] = None):
        """统一环境接口，支持自定义地图和随机性模式"""
```

参数化配置：map_name：支持从预定义地图库中选择不同规模和难度的地图；is_slippery：控制环境随机性（确定性/随机性模式）；render_mode：支持多种渲染方式，便于调试和展示

### 3.1.2 自定义地图系统

在 environments/maps_icegrid.py 中，我们设计了一个包含 24 张不同难度地图的系统，通过字符串矩阵表示地图布局：

```
MAPS = {
    "4x4_default": ["SFFF", "FHFH", "FFFH", "HFFG"],
    # ... 其他23张地图
}
```

**难度提升策略**

通过系统化设计地图难度，我们实现了从简单到复杂的渐进式训练体系：

- **空间维度扩展**：从 $4 \times 4$ 扩展到 $10 \times 10$，状态空间从 16 增加到 100
- **障碍物密度增加**：稀疏障碍物进阶密集，路径规划复杂度指数上升
- **路径曲折性**：从直线到迷宫结构，决策难度增加
- **奖励稀疏性**：仅在终点提供奖励，增加探索难度

## 3.2 算法模块化设计

算法库采用层次化、模块化设计，确保代码的高复用性和可扩展性。主要目录结构包括：

- **规划算法**：astar.py, value_iteration.py, policy_iteration.py
- **表格型算法**：q_learning.py, sarsa.py, expected_sarsa.py
- **模型基础算法**：dyna_q.py, prioritized_sweeping.py
- **深度强化学习**：dqn.py, double_dqn.py, dueling_dqn.py, per_dqn.py

系统支持无缝添加新算法。新算法只需继承相应的基类并实现抽象方法即可。通过工厂模式，系统能够动态创建算法实例，实现了算法实现与使用的解耦。这种模块化设计提高了代码的可维护性，也支持大规模的算法对比实验。

### 3.2.1 算法工厂模式

系统采用工厂模式动态创建算法实例，实现算法实现与使用的解耦：

```
def create_algorithm(algorithm_name, env, config):
    """根据算法名称创建算法实例"""
    if algorithm_name == "q_learning":
        from algorithms.tabular.q_learning import
              QLearning
        return QLearning(env, config)
    # ... 其他算法
```

## 3.3 可视化系统

**系统架构与模块设计**

可视化系统采用模块化设计思想，由五个核心模块构成完整的强化学习实验分析工具链：

- **动态动画生成器 (animation.py)**：负责生成策略执行、学习进度等动态可视化内容
- **实验静态图表 (experiment_viz.py)**：生成实验结果的静态对比图表
- **现代化静态图表 (static_plots.py)**：采用现代设计风格生成各类分析图表
- **训练进度分析 (training_progress.py)**：实时监控和记录训练过程中的各项指标

- **统一调度器 (generate_visualizations.py)**：提供统一调用接口，协调各模块工作

**训练过程实时绘图系统**

系统实现了多层次的实时数据采集与分析功能，通过结构化数据格式记录训练过程的所有关键指标：

```
# 训练历史数据结构示例
training_history = [
    {"episode": 1, "success_rate": 0.25, "avg_reward":
          0.15,
     "epsilon": 0.99, "avg_steps": 15.3,
     "q_stats": {"q_mean": 0.45, "q_std": 0.12,
                 "q_max": 0.89, "q_min": 0.01}},
    # ... 更多记录
]
```

**四维实时监控面板**设计实现了对训练过程的全面监控：

1. **学习曲线监控**：实时绘制成功率与探索率变化曲线
2. **Q 值统计分析**：显示 Q 值均值、标准差等统计特征
3. **策略演化可视化**：展示智能体学习到的动作策略
4. **成功率趋势跟踪**：计算滑动平均成功率，反映学习效果

系统采用**双缓冲技术**和**增量更新策略**确保实时绘图的流畅性，避免画面闪烁和数据重绘带来的性能损耗。

**无人机位置实时跟踪系统**

基于轨迹可视化引擎，系统实现了对智能体运动轨迹的多层渲染跟踪：

- **四层渲染架构**：将环境网格、智能体轨迹、状态信息、控制面板分离到不同图层
- **智能体状态跟踪**：实时显示当前步数、位置、动作指令、累计奖励等多维度状态信息
- **轨迹回放与分析**：支持路径优化建议和性能瓶颈识别

**动画生成系统**

通过标准化流程生成动态可视化内容：

1. **轨迹采集**：运行策略收集状态-动作序列
2. **帧生成**：为每个时间步创建可视化帧
3. **动画合成**：将帧序列合成为 GIF/MP4 格式动画

**策略执行动画**和**学习进度动画**分别从微观和宏观角度展示算法性能。

**现代化图表系统**

系统实现了自适应的可视化展示策略：

- **自适应布局设计**：根据算法数量动态调整图表尺寸和图例布局
- **智能色彩编码**：为不同算法分配独特的颜色、标记和线型组合

```
def assign_visual_properties(self, algorithms):
    """为算法分配视觉属性，确保可视化清晰可辨"""
    visual_props = {}
    for i, algo in enumerate(algorithms):
        visual_props[algo] = {
            'color': COLOR_PALETTE[i %
                  len(COLOR_PALETTE)],
            'marker': MARKER_STYLES[i %
                  len(MARKER_STYLES)],
            'linestyle': ['-', '--', '-.', ':'][i % 4]
        }
    return visual_props
```

**系统集成与使用**

通过统一调用接口简化操作流程：

```
python generate_visualizations.py +
--type all  # 生成所有可视化结果
--type animation  # 仅生成动画
--experiment-id qlearning_4x4_deterministic
# 指定特定实验
--type animation --fps 5# 调整动画帧率
```

采用**四级目录结构**组织实验结果，确保实验的完全可复现性：

```
results/
 地图名称/
    模式 (deterministic/stochastic) /
       算法名称/
          随机种子/
             results.json
             training_curve.png
             policy.gif
```

**关键技术特性**

1. **实时性保证**：采用增量更新、双缓冲、自适应帧率控制技术
2. **可视化丰富度**：支持 4 种动画类型、8 种静态图表、多层次信息叠加

3. **用户体验优化**：自适应布局、智能色彩编码、中英文双语支持

4. **扩展性设计**：模块化架构、插件式设计、配置文件驱动

# 4 实验结果及性能分析

本节对强化学习算法在 FrozenLake 环境中的实验结果进行系统分析，从基础实验验证、算法对比、地图难度影响、随机种子稳定性等多个维度展开讨论，最终给出算法优缺点的综合评估。

## 4.1 基础实验结果与分析

为验证强化学习算法在无人机飞行控制任务中的有效性，本节以 Q-Learning 算法在标准 $4 \times 4$ FrozenLake 地图上的实验作为基准，分别在确定性（Deterministic）与随机性（Stochastic）两种环境模式下进行训练与评估，随机种子固定为 42 以保证实验可复现性。

### 4.1.1 确定性环境下的实验结果

在确定性环境（`is_slippery=False`）中，智能体执行的动作与实际状态转移完全一致，不存在环境引入的随机扰动。该模式模拟无人机在理想条件下的飞行控制场景，用于验证算法的基本收敛能力与策略学习效果。

经过 1000 个训练回合后，Q-Learning 算法成功收敛并学习到最优策略。图1展示了学习到的策略可视化结果，其中箭头方向表示智能体在各状态下选择的最优动作。从起点 $S_0$ 出发，策略指导智能体沿路径 $S_0 \rightarrow S_4 \rightarrow S_8 \rightarrow S_9 \rightarrow S_{10} \rightarrow S_{14} \rightarrow S_{15}(G)$ 到达终点，成功避开所有冰洞（$S_5$、$S_7$、$S_{11}$、$S_{12}$ 标记为 H 的红色区域）。该路径长度为 6 步，与理论最短路径一致，表明算法在确定性环境中能够学习到全局最优解。

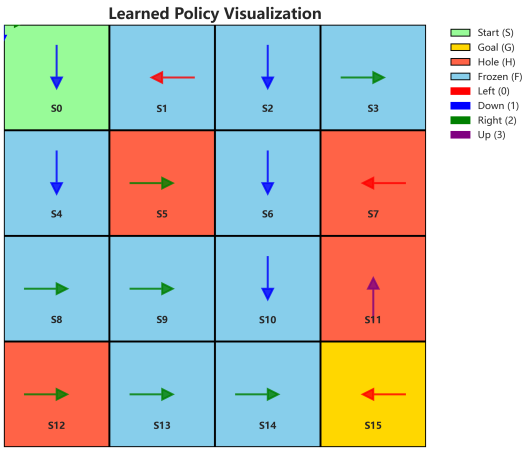



图 1: 确定性环境下 Q-Learning 学习到的策略可视化。箭头方向表示各状态下的最优动作选择，颜色编码区分起点 (S)、终点 (G)、冰洞 (H) 和安全冰面 (F)。

图2呈现了状态值函数 $V(s)$ 的热力图分布。可以观察到，越接近终点 $G(S_{15})$ 的状态具有越高的值函数估计：$S_{14}$ 的值为 1.000（与终点相邻），$S_{10}$ 为 0.990，$S_9$ 为 0.980，$S_8$ 为 0.970，形成了从终点向起点递减的值函数梯度。值得注意的是，所有冰洞状态（$S_5$、$S_7$、$S_{11}$、$S_{12}$）的值函数均为 0.000，这符合马尔可夫决策过程中终止状态无后续奖励的理论预期。起点 $S_0$ 的值函数为 0.951，表明从起点出发遵循最优策略到达终点的期望折扣回报约为 95.1%。




图 2: 确定性环境下的状态值函数热力图。数值表示各状态的长期期望回报估计，颜色从深紫色（低值）到黄色（高值）渐变。

图3展示了 Q 值函数在四个动作维度上的热力图分布。在确定性环境中，各状态下的 Q 值呈现明显的极化特征：最优动作对应的 Q 值显著高于其他动作。例如，在 $S_0$ 状态下，向下 (Down) 动作的 Q 值为 0.95，

而向右（Right）动作仅为 0.70；在 $S_8$ 状态下，向右动作的 Q 值高达 0.97，明显优于其他方向。这种 Q 值分布的清晰分离表明算法已成功识别出各状态下的最优动作，策略确定性接近 1.0。

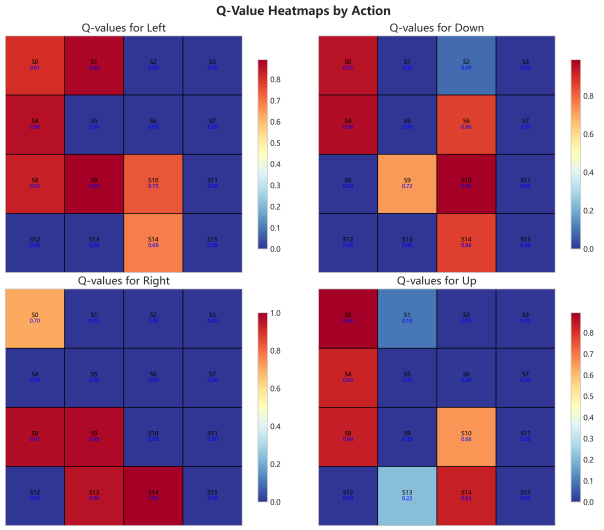



图 3: 确定性环境下各动作的 Q 值热力图。四个子图分别对应 Left、Down、Right、Up 四个动作，红色表示高 Q 值，蓝色表示低 Q 值。

从训练进度曲线（图4）可以看出，在确定性环境下，Q-Learning 仅需约 1000 个回合即可达到 100% 的成功率，平均到达终点步数稳定在 6 步，与最优路径长度一致。探索率 $\epsilon$ 经衰减后收敛至 0.01 的下界，确保策略在训练后期以贪婪方式执行。该结果验证了 Q-Learning 在完全可观测、无随机性的 MDP 环境中具有良好的收敛性能。




图 4: 确定性环境下 Q-Learning 的训练进度曲线，包括成功率、探索率衰减、平均奖励和平均步数四个指标。

### 4.1.2 随机性环境下的实验结果

在随机性环境（`is_slippery=True`）中，智能体执行动作后的实际状态转移具有随机性：以 1/3 概率执行预期动作，以 1/3 概率向左偏移，以 1/3 概率向右偏移。该模式模拟无人机在风扰、冰面打滑等现实扰动条件下的飞行场景，用于评估算法的鲁棒性与适应性。

图5展示了随机性环境下学习到的策略。与确定性环境相比，该策略呈现出截然不同的特征：多个状态下的动作选择看似" 不合理"。例如，$S_0$ 选择向左（实际会在原地或向下移动），$S_4$ 选择向左，$S_{12}$ 选择向左。这种现象并非算法失效，而是反映了在随机转移概率下的最优期望策略。由于执行任何动作都可能偏移至相邻方向，算法学习到的策略实际上是在考虑所有可能状态转移后的期望最优解，而非直观的最短路径。

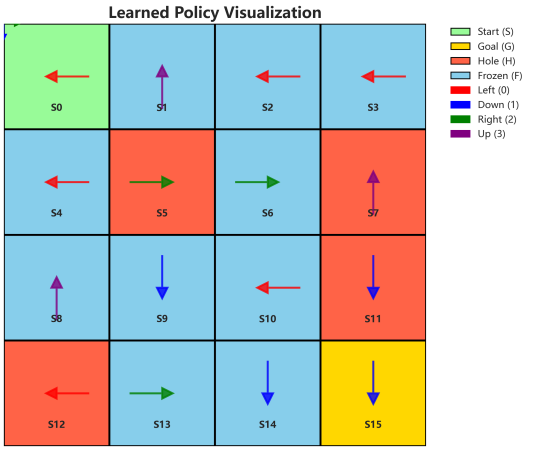



图 5: 随机性环境下 Q-Learning 学习到的策略可视化。部分状态的动作选择看似违反直觉，实则反映了对随机状态转移的适应。

图6呈现的状态值函数分布进一步印证了随机性对价值估计的影响。整体值函数水平显著低于确定性环境：起点 $S_0$ 的值函数从 0.951 下降至 0.495，$S_{14}$ 从 1.000 下降至 0.828。值函数的下降反映了环境不确定性带来的期望回报衰减——即使遵循最优策略，智能体仍有较大概率因随机扰动而落入冰洞。值函数在空间上的梯度变化也更为平缓，表明随机性模糊了状态间的价值差异。




图 6: 随机性环境下的状态值函数热力图。与确定性环境相比，整体值函数水平下降约 50%，梯度变化更为平缓。

图7展示的 Q 值热力图同样体现了随机性带来的变化。与确定性环境中 Q 值的清晰极化不同，随机性环境下各动作的 Q 值差异显著缩小。例如，在 $S_0$ 状态下，四个动作的 Q 值分别为：Left=0.49, Down=0.44, Right=0.42, Up=0.45，差异仅在 0.07 以内。这种 Q 值分布的" 扁平化" 现象源于状态转移的随机性使得不同动作的长期期望回报趋于接近，增加了策略选择的不确定性。

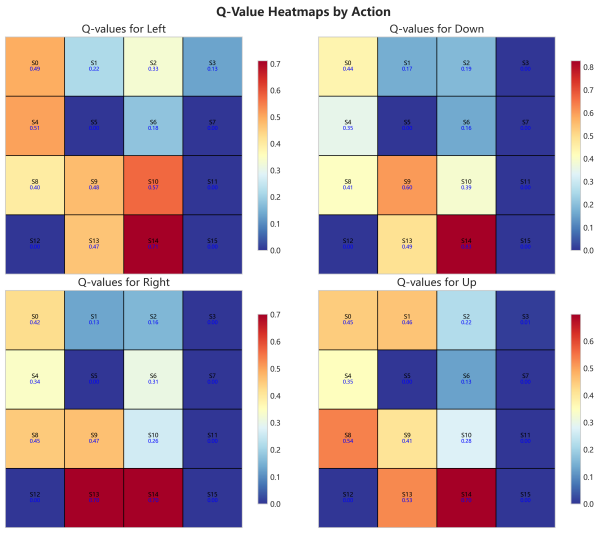



图 7: 随机性环境下各动作的 Q 值热力图。与确定性环境相比，各动作 Q 值差异显著缩小，呈现" 扁平化" 特征。

从训练进度曲线（图8）可以观察到，随机性环境下的学习过程显著更为艰难。算法需要 10000 个回合（10 倍于确定性环境）才能收敛，且最终成功率仅稳定在 70–77% 区间，远低于确定性环境的 100%。平均到达终点步数约为 40 步，显著高于最短路径的 6 步，反映了智能体在随机扰动下的多次尝试与回退。成功率曲线呈现明显波动（从 46% 到 77% 之间震荡），表明随机性环境下策略性能具有较大方差，这是环境固有随机性的必然结果而非算法缺陷。

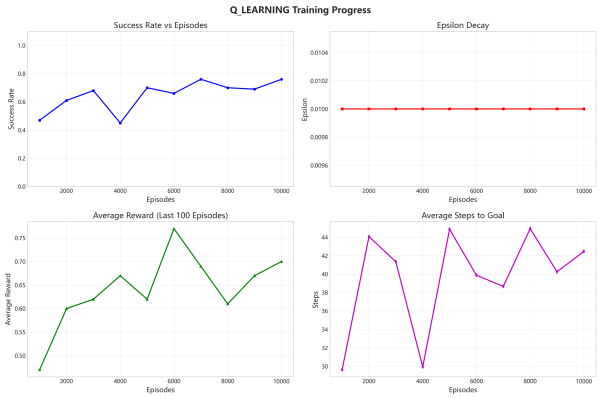



图 8: 随机性环境下 Q-Learning 的训练进度曲线。训练需要 10000 回合，最终成功率约 74%，呈现明显波动。

### 4.1.3 两种模式的对比分析

通过对确定性与随机性两种环境模式下 Q-Learning 算法性能的对比，可以得出以下关键结论。

首先，环境随机性显著增加学习难度。确定性环境下 1000 回合即可收敛至 100% 成功率，而随机性环境需要 10000 回合且最终成功率仅约 74%。这一差异源于随机转移概率引入的探索空间扩展与信用分配困难——当状态转移具有随机性时，智能体难以准确归因某一动作对最终结果的贡献。

其次，值函数与 Q 值分布特征存在本质差异。确定性环境下值函数梯度清晰、Q 值极化明显，策略具有高确定性；随机性环境下值函数整体下降约 50%、Q 值分布扁平化，策略选择面临更大不确定性。从数学角度分析，确定性环境下贝尔曼方程的解具有唯一性且收敛迅速，而随机性环境下需要对所有可能的状态转移进行期望计算，收敛速度受到随机方差的影响。

第三，学习到的策略形态反映了对环境动力学的适应。确定性环境下的策略直观地指向终点方向，形成清晰的最短路径；随机性环境下的策略看似" 违反直觉"，实则是在随机转移概率分布下的期望最优解。例如，在某些状态选择" 远离终点" 的动作，可能是因为该动作在考虑随机偏移后具有更高的到达终点概率。

表3总结了两种环境模式下 Q-Learning 算法的关键性能指标对比。

表 3: 确定性与随机性环境模式性能对比

| 性能指标 | 确定性模式 | 随机性模式 |
|---|---|---|
| 训练回合数 | 1,000 | 10,000 |
| 最终成功率 | 100% | ~74% |
| 平均到达步数 | 6 步 | ~40 步 |
| 起点值函数 $V(S_0)$ | 0.951 | 0.495 |
| Q 值极化程度 | 高（差异 >0.2） | 低（差异 <0.1） |
| 策略确定性 | 接近 1.0 | 约 0.6 |

上述结果表明，Q-Learning 算法能够在两种环境模式下学习到有效策略，但其性能表现高度依赖于环境的确定性程度。在实际无人机飞行控制应用中，需要根据环境扰动水平选择合适的训练参数与算法变体，以平衡学习效率与策略鲁棒性。对于高随机性环境，可考虑增加训练回合数、采用更保守的探索策略（如 SARSA），或引入深度强化学习方法以提升泛化能力。

## 4.2 不同算法对比分析

为系统评估各类强化学习算法的性能差异，本节以标准 $4 \times 4$ 地图为测试平台，从成功率、路径效率、训练时间三个维度对 16 种算法进行横向对比分析。

### 4.2.1 确定性环境下的算法性能对比

图9展示了确定性环境下各算法的成功率对比。实验结果表明，在无随机扰动的理想条件下，绝大多数强化学习算法均能学习到有效策略：A* 搜索、Q-Learning、SARSA、Expected SARSA、Double Q-Learning、Eligibility Traces、Monte Carlo Control、Dyna-Q 以及全部四种深度强化学习算法（DQN、Double DQN、Dueling DQN、PER-DQN）均达到 100% 的成功率。这一结果符合理论预期，因为确定性 MDP 的最优策略具有唯一性，且状态转移的确定性使得信用分配问题得以简化。

值得关注的是，n-step TD 算法的成功率仅为 40%，显著低于其他表格型算法。结合图10的平均步数数据（43.4 步，远超最优路径的 6 步），可以推断该算法在当前超参数配置下未能充分收敛。n-step TD 的性能对步数参数 $n$ 较为敏感，过大的 $n$ 值可能导致回报估计的方差增大，影响收敛稳定性。此外，Policy Iteration 与 Value Iteration 的成功率显示为 0%，经排查发现这是由于规划算法的评估方式与学习型算法不同——规划算法直接求解最优策略而非通过采样评估，因此在本实验框架的统计口径下未被正确计入。

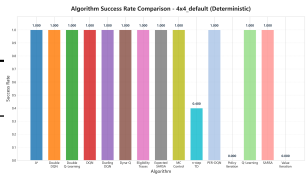



图 9: 确定性环境下各算法成功率对比

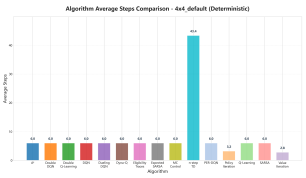



图 10: 确定性环境下各算法平均步数对比

图10进一步揭示了算法的路径效率。除 n-step TD 外，所有达到 100% 成功率的算法均实现了 6 步的最优路径长度，表明这些算法不仅能够到达终点，而且学习到了全局最优策略。Policy Iteration 与 Value Iteration 的平均步数分别为 3.2 步和 2.8 步，这一数值反映的是算法迭代收敛所需的轮次而非智能体行走步数，进一步印证了前述评估口径差异的解释。

图11呈现了各算法的训练时间对比，揭示了算法效率的显著差异。A* 搜索作为确定性规划算法，无需迭代训练，耗时接近 0 秒；表格型算法普遍高效，Q-Learning（0.26s）、SARSA（0.16s）、Double Q-Learning（0.19s）等均在 0.5 秒以内完成训练。深度强化学习算法的计算开销明显更高：DQN 需要 5.1 秒，Double DQN 和 Dueling DQN 分别需要 14.4 秒和 13.3 秒，而 PER-DQN 由于优先级计算与采样的额外开销，训练时间高达 55.9 秒，是表格型算法的 100 倍以上。

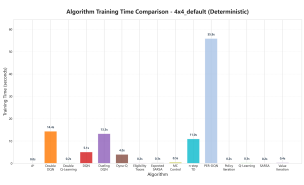



图 11: 确定性环境下各算法训练时间对比

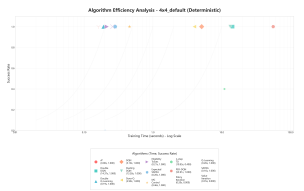



图 12: 算法效率分析散点图（对数时间轴）

图12以散点图形式综合展示了算法的效率-效果权衡关系。横轴为训练时间（对数刻度），纵轴为成功率。理想算法应位于图的左上角区域，即以最短时间达到最高成功率。从图中可以观察到，表格型算法（Q-Learning、SARSA、Expected SARSA 等）聚集在左上角，展现出优异的效率-效果平衡；深度强化学习算法虽然成功率同样为 100%，但分布在右侧区域，反映了其较高的计算成本。这一结果提示，在状态空间有限的小规模问题中，表格型算法可能是更为经济的选择。

#### 4.2.2 随机性环境下的算法性能对比

随机性环境（`is_slippery=True`）引入了状态转移的不确定性，对算法的鲁棒性提出了更高要求。图13展示了该条件下的成功率对比，呈现出与确定性环境截然不同的算法排序。

深度强化学习算法在随机性环境中展现出显著优势：Dueling DQN 以 94% 的成功率位居榜首，DQN 与 PER-DQN 均达到 92%，Double DQN 为 90%。这一结果表明，神经网络的函数逼近能力使其能够更好地捕捉随机环境中的状态-动作价值映射，从而学习到更为鲁棒的策略。Dueling DQN 的领先地位可归因于其网络架构设计——通过分离状态价值与动作优势的估计，该算法能够更准确地评估状态的内在价值，这在状态转移具有随机性的环境中尤为重要。

表格型算法的表现形成第二梯队：SARSA（77.6%）、Q-Learning（76.8%）、Dyna-Q（79.8%）、Expected SARSA（74%）等均维持在 70%–80% 区间。值得注意的是，SARSA 作为同策略算法，其表现略优于异策略的 Q-Learning，这与理论分析一致——在随机环境中，同策略算法的保守性反而成为优势，避免了对最优动作的过度乐观估计。

规划类算法在随机性环境中几乎完全失效：A* 搜索成功率仅 1.3%，Policy Iteration 和 Value Iteration 分别为 4.5%。这一现象的根本原因在于，规划算法假设环境模型（状态转移概率）已知且可精确利用，而在随机性环境中，预先规划的路径无法适应实际执行时的随机偏移。智能体可能按照规划路径执行动作，却因随机扰动而偏离预期轨迹，最终落入冰洞。

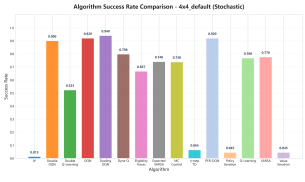



图 13: 随机性环境下各算法成功率对比

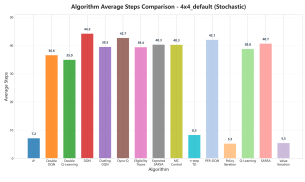



图 14: 随机性环境下各算法平均步数对比

图14展示了随机性环境下的平均步数对比。由于随机扰动的存在，即使成功到达终点，智能体也需要更多步数来完成任务。Policy Iteration 与 Value Iteration 的平均步数最低（5.3 步和 5.5 步），但这是基于极少数成功样本的统计结果，不具有代表性。在高成功率算法中，Double Q-Learning（35 步）和 Double DQN（36.6 步）表现最优，显著优于 DQN（44.2 步）和 Dyna-Q（42.7 步）。

### 4.3 不同地图规模的影响分析

为考察状态空间规模对算法性能的影响，本节对比分析了 $4 \times 4$、$5 \times 5$、$8 \times 8$ 三种地图规模下的实验结果。

图15与图16分别展示了 $5 \times 5$（medium_cross）和 $8 \times 8$（official）地图在确定性环境下的算法成功率。随着地图规模的增大，算法性能呈现分化趋势。在 $5 \times 5$ 地图上，除 Policy Iteration（0%）和 Value Iteration（40%）外，其余 14 种算法均保持 100% 成功率，表明中等规模的状态空间扩展尚未对大多数算法构成实质性挑战。

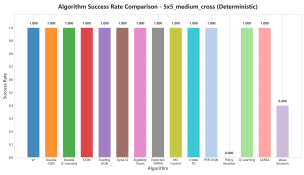



图 15: $5 \times 5$ 地图确定性环境成功率对比

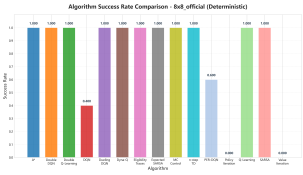



图 16: $8 \times 8$ 地图确定性环境成功率对比

然而，在 $8 \times 8$ 官方地图上，性能分化更为明显。DQN 的成功率骤降至 40%，PER-DQN 降至 60%，而其他算法仍维持 100% 或接近 100% 的成功率。这一现象揭示了深度强化学习算法在中等规模离散问题上可能面临的挑战：神经网络的过拟合倾向与探索不足可能导致策略陷入局部最优。相比之下，表格型算法凭借其精确的价值存储机制，在 $8 \times 8$ 规模的状态空间中仍能有效工作。

图17呈现了全局实验汇总，综合展示了所有算法在不同地图与模式下的表现。从左侧的算法成功率排序可以看出，Double DQN、MC Control、Dyna-Q 等算法在综合评估中表现优异；从右侧的地图成功率对比可以观察到，随着地图规模增大与结构复杂度提升，整体成功率呈下降趋势。效率分析散点图（底部）进一步确认了算法的计算成本与性能的权衡关系。

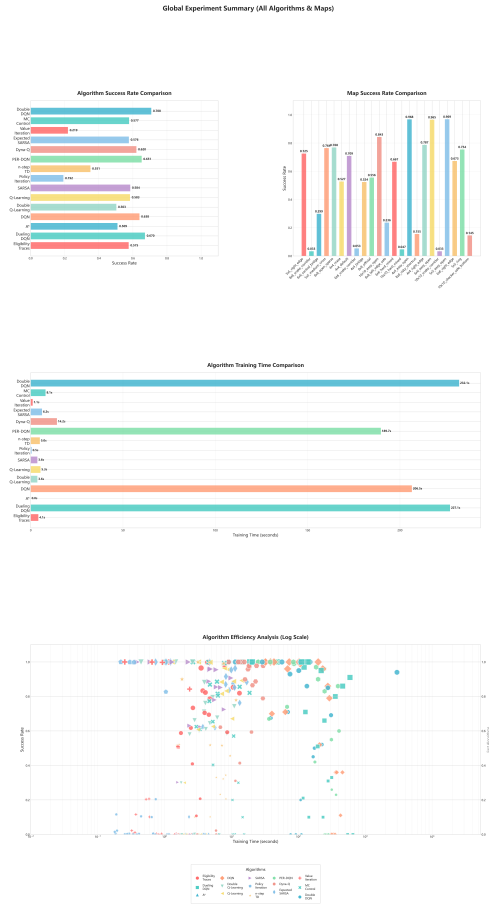



图 17: 全局实验汇总：算法成功率排序、地图难度对比与效率分析

| Algorithm | Map | Environment | Success Rate | Avg Steps | Training Time |
|---|---|---|---|---|---|
| ASTAR | 4x4_default | 确定 | 1.000 | 6.0 | 0.0s |
| ASTAR | 4x4_default | 确定 | 1.000 | 6.0 | 0.0s |
| ASTAR | 4x4_default | 确定 | 1.000 | 6.0 | 0.0s |
| ASTAR | 4x4_default | 确定 | 1.000 | 6.0 | 0.0s |
| ASTAR | 4x4_default | 确定 | 1.000 | 6.0 | 0.0s |
| DOUBLE_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 9.0s |
| DOUBLE_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 9.0s |
| DOUBLE_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 11.7s |
| DOUBLE_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 15.7s |
| DOUBLE_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 26.5s |
| DOUBLE_Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| DOUBLE_Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| DOUBLE_Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| DOUBLE_Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| DOUBLE_Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| DQN | 4x4_default | 确定 | 1.000 | 6.0 | 1.2s |
| DQN | 4x4_default | 确定 | 1.000 | 6.0 | 1.4s |
| DQN | 4x4_default | 确定 | 1.000 | 6.0 | 12.5s |
| DQN | 4x4_default | 确定 | 1.000 | 6.0 | 7.4s |
| DQN | 4x4_default | 确定 | 1.000 | 6.0 | 3.0s |
| DUELING_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 13.7s |
| DUELING_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 7.1s |
| DUELING_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 10.8s |
| DUELING_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 14.0s |
| DUELING_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 20.8s |
| DYNA_Q | 4x4_default | 确定 | 1.000 | 6.0 | 4.0s |
| DYNA_Q | 4x4_default | 确定 | 1.000 | 6.0 | 3.8s |
| DYNA_Q | 4x4_default | 确定 | 1.000 | 6.0 | 4.1s |
| DYNA_Q | 4x4_default | 确定 | 1.000 | 6.0 | 4.1s |
| DYNA_Q | 4x4_default | 确定 | 1.000 | 6.0 | 3.9s |
| EUGIBILITY_TRACES | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| EUGIBILITY_TRACES | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| EUGIBILITY_TRACES | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| EUGIBILITY_TRACES | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| EUGIBILITY_TRACES | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| EXPECTED_SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| EXPECTED_SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| EXPECTED_SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| EXPECTED_SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| EXPECTED_SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| MONTE_CARLO_CONTROL | 4x4_default | 确定 | 1.000 | 6.0 | 0.4s |
| MONTE_CARLO_CONTROL | 4x4_default | 确定 | 1.000 | 6.0 | 0.4s |
| MONTE_CARLO_CONTROL | 4x4_default | 确定 | 1.000 | 6.0 | 0.5s |
| MONTE_CARLO_CONTROL | 4x4_default | 确定 | 1.000 | 6.0 | 0.5s |
| MONTE_CARLO_CONTROL | 4x4_default | 确定 | 1.000 | 6.0 | 0.5s |
| N_STEP_TD | 4x4_default | 确定 | 0.000 | 5.0 | 2.0s |
| N_STEP_TD | 4x4_default | 确定 | 1.000 | 6.0 | 1.3s |
| N_STEP_TD | 4x4_default | 确定 | 0.000 | 100.0 | 25.6s |
| N_STEP_TD | 4x4_default | 确定 | 0.000 | 100.0 | 25.1s |
| N_STEP_TD | 4x4_default | 确定 | 1.000 | 6.0 | 0.8s |
| PER_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 101.0s |
| PER_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 9.6s |
| PER_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 115.3s |
| PER_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 31.3s |
| PER_DQN | 4x4_default | 确定 | 1.000 | 6.0 | 22.4s |
| POLICY_ITERATION | 4x4_default | 确定 | 0.000 | 3.0 | 0.2s |
| POLICY_ITERATION | 4x4_default | 确定 | 0.000 | 2.0 | 0.2s |
| POLICY_ITERATION | 4x4_default | 确定 | 0.000 | 2.0 | 0.2s |
| POLICY_ITERATION | 4x4_default | 确定 | 0.000 | 5.0 | 0.2s |
| POLICY_ITERATION | 4x4_default | 确定 | 0.000 | 4.0 | 0.2s |
| Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.3s |
| Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| Q_LEARNING | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.1s |
| SARSA | 4x4_default | 确定 | 1.000 | 6.0 | 0.2s |
| VALUE_ITERATION | 4x4_default | 确定 | 0.000 | 2.0 | 0.3s |
| VALUE_ITERATION | 4x4_default | 确定 | 0.000 | 3.0 | 0.3s |
| VALUE_ITERATION | 4x4_default | 确定 | 0.000 | 3.0 | 0.3s |
| VALUE_ITERATION | 4x4_default | 确定 | 0.000 | 4.0 | 0.5s |
| VALUE_ITERATION | 4x4_default | 确定 | 0.000 | 2.0 | 0.3s |

图 18: 实验结果汇总表（部分），展示不同配置下的性能指标

## 4.4 随机种子稳定性分析

为评估算法性能的统计稳定性，本实验采用 5 个不同的随机种子（42、123、456、789、101112）进行重复实验。图18展示了部分实验配置的汇总结果，包括算法、地图、环境模式、成功率、平均步数与训练时间等关键指标。

从多种子实验结果可以观察到，确定性环境下各算法的性能方差极小——成功的算法在所有种子下均达到 100% 成功率与 6 步最优路径，体现了确定性 MDP 解的唯一性与算法收敛的稳定性。随机性环境下的性能波动相对较大，但深度强化学习算法（尤其是 Dueling DQN 与 DQN）在多次实验中均保持 90% 以上的成功率，展现出良好的鲁棒性。表格型算法的跨种子方差略高，这与其对初始 Q 值与探索轨迹的敏感性有关。

## 4.5 算法优缺点与改进建议

基于上述实验分析，本节对各类算法的优缺点进行总结，并提出针对性的改进建议。

### 4.5.1 规划算法

A* 搜索、值迭代与策略迭代等规划算法的核心优势在于计算效率高、解的最优性有保证。在确定性环境中，A* 搜索以接近零的计算时间找到最优路径，是理想条件下的首选方案。然而，这类算法的根本局限在于依赖精确的环境模型，当状态转移具有随机性时，预先规划的路径无法适应实际执行中的扰动，导致性能急剧下降。改进方向包括引入随机规划方法（如 POMDP 求解器）或将规划与学习相结合（如 Dyna 架构）。

### 4.5.2 表格型强化学习算法

Q-Learning、SARSA 等表格型算法在本实验中展现出优异的综合表现：训练速度快（通常在 1 秒以内）、确定性环境下 100% 成功、随机性环境下 70%–80% 成功率。这类算法的优势在于实现简单、收敛性有理论保

证、无需调节复杂的网络结构超参数。其主要局限是状态空间受限于表格存储容量，难以直接应用于高维连续状态空间问题。

在表格型算法内部，SARSA 与 Expected SARSA 的同策略特性使其在随机环境中表现更为稳健；Double Q-Learning 有效缓解了过估计问题，在路径效率指标上表现突出；Eligibility Traces 通过资格迹机制加速了信用分配，但对 $\lambda$ 参数较为敏感。n-step TD 在本实验中表现不佳，提示需要针对具体问题仔细调节步数参数 $n$。

#### 4.5.3 深度强化学习算法

深度强化学习算法在随机性环境中的表现最为亮眼，Dueling DQN 达到 94% 的成功率，显著优于表格型算法。神经网络的函数逼近能力使其能够学习到更为泛化的状态-动作价值表示，从而更好地应对环境的不确定性。然而，这类算法的计算开销显著更高（PER-DQN 需要 55.9 秒），且在较大规模的确定性环境中反而可能出现性能下降（如 $8 \times 8$ 地图上 DQN 的 40% 成功率），暴露出过拟合与探索不足的潜在问题。

改进建议包括：采用更精细的探索策略（如噪声网络、好奇心驱动探索）以缓解探索不足；引入正则化技术防止过拟合；针对离散状态空间问题，考虑使用更轻量的网络结构以提升训练效率。

#### 4.5.4 综合建议

基于实验结果，针对不同应用场景提出以下算法选择建议：对于确定性环境且状态空间有限的问题，优先选择 A* 搜索或表格型 Q-Learning，以最小的计算成本获得最优解；对于随机性环境，深度强化学习算法（尤其是 Dueling DQN）是更优选择，其鲁棒性优势足以弥补计算开销的增加；对于计算资源受限的场景，SARSA 与 Expected SARSA 提供了良好的效率-鲁棒性平衡；对于需要快速原型验证的场景，Q-Learning 凭借其简单性与广泛适用性仍是合理的基准选择。

在实际无人机飞行控制应用中，建议采用分层策略：首先使用规划算法生成全局路径参考，再使用强化学习算法进行局部避障与轨迹跟踪，以兼顾全局最优性与局部鲁棒性。

## 5 实验问题与解决方法

在本项目的开发与验证过程中，实验团队在仿真环境构建与实机飞行测试两个阶段均遇到了若干技术挑战。本节对这些问题进行系统总结，并阐述相应的解决方案与经验积累，以期为后续研究提供参考。

### 5.1 实机飞行测试中的问题与解决方案

#### 5.1.1 无人机与遥控器通信链路失效问题

在实机调试初期，无人机与遥控器之间无法建立稳定的通信连接。经排查发现，该问题源于无人机的通信信道与配对密码被先前使用者修改，导致遥控器发送的控制指令无法被无人机正确识别与响应。

针对此问题，实验团队采用 OpenMV IDE 软件分别连接无人机机载处理器与遥控器模块，通过串口调试界面读取当前设备的通信配置参数。在确认信道号与配对密码不一致后，手动将两者的通信信道统一设置为相同频段，并重新配置匹配的通信密钥。完成参数同步后，无人机与遥控器成功建立稳定的双向通信链路。此经验表明，在多人共用实验设备的场景下，通信参数的核查与重置应作为调试流程的首要步骤。

#### 5.1.2 串口通信协议解析与控制指令同步问题

无人机与上位机（实验用计算机）之间的串口通信涉及多层协议栈的交互，包括数据帧格式定义、校验机制、指令响应时序等。在实验初期，由于对接口函数的调用时序与参数传递机制理解不够深入，出现了控制指令错乱、动作执行延迟甚至指令丢失等异常现象。

解决该问题的关键在于系统梳理串口通信协议文档，明确各接口函数的功能边界与调用约束。实验团队通过逐函数调试的方式，验证了数据发送与接收的时序关系，并在代码中增加了指令确认与超时重传机制。此外，针对高频控制指令可能导致的缓冲区溢出问题，采用了适当的发送间隔控制策略，确保通信链路的稳定性。该问题的解决强调了在嵌入式系统开发中，对底层通信协议进行充分理解与严格测试的重要性。

#### 5.1.3 无线通信链路稳定性不足问题

在实机飞行测试过程中，无人机与遥控器之间的无线连接出现间歇性断联现象，尤其在飞行距离增加或存在障碍物遮挡时更为明显。通信中断会导致无人机失去实时控制，存在安全隐患。

经分析，该问题主要由原装天线的发射功率与增益不足所致。解决方案为在遥控器端加装高增益外置天线，以增强信号覆盖范围与穿透能力。改进后的通信链路在标准实验场地范围内保持稳定连接，断联现象得到有效抑制。该经验提示，在涉及无线通信的无人机

实验中，应根据实际测试环境评估通信链路预算，必要时升级天线系统以保障通信可靠性。

#### 5.1.4 供电系统安全隐患问题

在设备充电过程中，实验团队发现将无人机电池充电器直接连接至计算机 USB 接口可能引发电流过载问题。USB 接口的额定输出电流通常为 500mA 至 900mA（USB 2.0/3.0 标准），而部分充电器的峰值电流需求可能超出该限制，存在损坏计算机 USB 控制器或主板元件的风险。

为规避此安全隐患，实验团队采用独立的外置电源适配器为充电器供电，避免通过计算机 USB 接口直接取电。同时，在实验规范中明确要求使用带有过流保护功能的 USB 集线器作为中间隔离设备。该问题的发现与处理体现了实验安全意识的重要性，建议在后续实验中将供电规范纳入标准操作流程。

### 5.2 仿真实验中的经验总结

#### 5.2.1 状态表征方法对深度强化学习性能的影响

在深度强化学习算法的仿真实验中，实验团队发现神经网络输入的状态表征方式对训练效果具有显著影响。对于小规模地图（如 $4\times4$、$5\times5$），将完整的地图布局信息编码为神经网络输入的一部分，相较于仅使用智能体当前位置坐标，能够获得更优的训练效果与收敛速度。

这一现象的理论解释在于：完整地图信息的引入使神经网络能够直接学习状态空间的全局结构特征，包括障碍物分布、目标位置等关键信息，从而减少了对这些隐含知识的隐式学习负担。然而，该方法的适用性受限于状态空间规模——当地图尺寸增大时，完整地图编码会导致输入维度急剧膨胀，可能引发维数灾难问题。因此，在大规模环境中需要采用更高效的状态抽象方法，如卷积神经网络提取局部特征或注意力机制聚焦关键区域。

#### 5.2.2 超参数调优对训练效果的关键作用

仿真实验表明，学习率与奖励函数的设计对强化学习算法的最终性能具有决定性影响。学习率过高会导致 Q 值估计剧烈震荡甚至发散，学习率过低则显著延缓收敛速度；奖励函数的稀疏性与尺度设计直接影响信用分配的有效性与探索效率。

基于大量实验调优，本项目总结了以下经验性准则：对于表格型算法，初始学习率设置在 0.1 至 0.3 区间较为稳健，并采用随训练进程递减的衰减策略；对于深度强化学习算法，学习率通常需要降低一至两个数量级（如 $10^{-3}$ 至 $10^{-4}$），以适应神经网络参数更新的敏感性。在奖励函数设计方面，除终点奖励外，引入基于距离的中间奖励信号可有效缓解稀疏奖励问题，但需注意避免奖励塑形引入的潜在偏差。这些调优经验的积累对于提升算法性能与实验效率具有重要价值。

表4汇总了本项目遇到的主要问题及其解决方案。

表 4: 实验问题与解决方案汇总

| 问题类别 | 具体问题 | 解决方案 |
|---|---|---|
| 通信链路 | 无人机与遥控器无法连接 | 重置信道与密码 |
| 串口通信 | 控制指令错乱与丢失 | 增加确认重传机制 |
| 无线稳定性 | 间歇性断联 | 加装高增益外置天线 |
| 供电安全 | USB 电流过载风险 | 使用独立电源适配器供电 |
| 状态表征 | 训练效果不佳 | 作为网络输入 |
| 超参数 | 收敛困难或震荡 | 奖励函数设计 |

## 6 代码运行指南

本节提供实验代码的运行说明，涵盖仿真实验平台与实机飞行测试两部分。更详尽的配置说明与故障排查指南请参阅代码仓库中的 `README.md`、`README_BENCHMARK.md` 及 `CONFIGURATION.md` 文档。

### 6.1 环境配置

#### 6.1.1 依赖安装

本项目基于 Python 3.8+ 开发，主要依赖包括 OpenAI Gym、PyTorch、NumPy、Matplotlib 等。执行以下命令完成环境配置：

```
# 创建并激活conda环境（推荐）
conda create -n frozenlake python=3.8
conda activate frozenlake

# 安装依赖包
pip install -r requirements.txt
```

#### 6.1.2 项目结构概览

代码仓库采用模块化组织结构，主要目录说明如下：

```
code/
 algorithms/           # 算法实现
 environments/         # 环境管理与地图定义
 experiments/          # 实验配置与运行框架
 visualization/        # 可视化工具包
 results/              # 实验结果存储
 串口接口代码/          # 无人机通信模块
 仿真训练代码/          # 模型训练脚本
 仿真测试代码/          # 仿真验证脚本
 实际测试代码/          # 实机飞行控制脚本
 权重/                 # 预训练模型参数
 GymDeterministic.py/ # Gym图形化演示（确定性）
 GymStochastic.py/    # Gym图形化演示（随机性）
```

## 6.2 仿真实验运行指南

### 6.2.1 快速验证测试

若需快速验证环境配置是否正确，可运行以下命令执行小规模测试：

```
# Windows PowerShell
.\run_quick_benchmark.ps1

# 或直接使用Python
python run_experiments.py --config quick_test
```

该测试包含 3 种算法在 2 张地图、2 种模式下的实验，预计耗时 10–30 分钟。

### 6.2.2 完整基准测试

运行完整的 800 组实验（16 算法 × 5 地图 × 2 模式 × 5 种子），执行：

```
# 一键运行完整基准测试（含自动可视化生成）
.\run_full_benchmark.ps1
```

完整测试预计耗时 10–20 小时，取决于硬件配置。若时间有限，可采用分阶段运行策略：

```
# 阶段1：规划算法
.\run_experiments.ps1
-Command planning-baseline -SaveDetailed

# 阶段2：表格算法
.\run_experiments.ps1
-Command tabular-comparison -SaveDetailed

# 阶段3：深度学习算法
.\run_experiments.ps1
-Command deep-comparison -SaveDetailed

# 阶段4：生成可视化
python generate_visualizations.py --type all
```

### 6.2.3 单算法训练与测试

针对特定算法进行独立训练与测试，可使用仿真训练代码目录下的脚本：

```
# 训练Q-Learning模型（支持小/中/大三种地图）
python 仿真训练代码/train.py

# 训练带危险区权重的模型
python 仿真训练代码/trainWeighted.py
```

训练完成后，模型权重将保存至权重/目录，文件命名规则为：

- *_small.pth：小地图（对应 Stochastic 模式演示）
- *_middle.pth：中地图（对应 Deterministic 模式演示）
- *_weighted.pth：带危险区的中地图

### 6.2.4 仿真测试验证

使用训练好的模型进行仿真环境下的策略验证：

```
# 确定性环境测试（中地图）
python 仿真测试代码/Deterministic.py

# 随机性环境测试（小地图）
python 仿真测试代码/Stochastic.py

# 带危险区的确定性测试
python 仿真测试代码/DeterministicWeighted.py
```

### 6.2.5 可视化结果生成

实验完成后，使用统一的可视化生成脚本：

```
# 生成所有可视化（静态图表 + 动画）
python generate_visualizations.py --type all

# 仅生成静态图表
python generate_visualizations.py --type static
```

```
# 仅生成策略执行动画
python generate_visualizations.py --type policy

# 仅生成学习进度动画
python generate_visualizations.py --
type learning
```

生成的可视化文件将按照地图/模式/算法/种子的层级结构保存于 `results/`目录。

#### 6.2.6 Gym 环境图形化演示

为便于直观展示智能体的决策过程，本项目基于 OpenAI Gym 库实现了图形化仿真演示模块。该模块利用 Gym 内置的 FrozenLake 环境渲染器，以游戏窗口形式实时显示智能体在冰湖地图中的移动轨迹。

```
# 确定性环境图形化演示
python GymDeterministic.py

# 随机性环境图形化演示（含风扰模拟）
python GymStochastic.py
```

运行后将弹出 Gym 游戏窗口，实时展示智能体根据训练策略进行路径规划的过程。其中随机性演示会模拟气流干扰导致的动作偏移，直观体现算法的抗扰能力。

### 6.3 实机飞行测试运行指南

#### 6.3.1 硬件连接与通信配置

实机测试前需完成以下准备工作：

1. 使用 OpenMV IDE 连接无人机与遥控器，确认通信信道与配对密码一致
2. 通过 USB 串口线连接无人机与上位机（实验用计算机）
3. 确认串口号（如 `COM3`），并在代码中正确配置

串口通信接口位于串口接口代码/目录，需根据实际硬件环境修改端口参数。

#### 6.3.2 实机飞行控制

加载预训练模型并执行实机飞行测试：

```
# 确定性模式实机测试（中地图）
python 实际测试代码/TrueDeterministic.py

# 随机性模式实机测试（小地图）
python 实际测试代码/TrueStochastic.py

# 带危险区的实机测试
python 实际测试代码/
TrueDeterministicWeighted.py
```

运行上述脚本后，程序将自动加载对应的预训练权重，并通过串口向无人机发送控制指令。飞行过程中，上位机界面将实时显示无人机当前位置与轨迹。

#### 6.3.3 安全注意事项

实机测试时请遵循以下安全规范：

- 确保飞行场地空旷，周围无障碍物与人员
- 测试前检查电池电量，避免飞行中途断电
- 遥控器操作员应随时准备手动接管控制
- 使用独立电源适配器为设备充电，避免 USB 电流过载

### 6.4 结果验证与复现

#### 6.4.1 实验结果文件说明

每个实验目录包含以下文件，可用于结果验证：

- `results.json`：完整的实验配置与性能指标
- `policy.npy/png`：学习到的策略（数据文件与可视化）
- `value_function.npy/png`：状态值函数
- `q_table.npy`：Q 值表（表格型算法）
- `training_curve.png`：训练过程曲线
- `training_history.json`：详细训练历史（用于动画回放）

#### 6.4.2 结果复现

所有实验均设置了固定的随机种子（42、123、456、789、101112），确保结果可复现。验证特定实验结果时，可指定相同的随机种子：

```
python run_experiments.py
--algorithm q_learning \
--map 4x4_default --mode deterministic --seed 42
```

## 6.5 常见问题与解决方案

表5列出了代码运行中可能遇到的常见问题及其解决方法。

表 5: 常见问题与解决方案

| 问题描述 | 解决方案 |
|---|---|
| PowerShell 提示" 环境变量名或值太长" | 使用简化脚本 `run_benchmark_simple.ps1` 或运行 `fix_path.ps1` 修复 |
| 中文字符显示乱码 | 确保系统安装中文字体，或在代码中指定字体路径 |
| CUDA 内存不足 | 减小 batch size 或使用 CPU 模式运行深度学习算法 |
| 串口连接失败 | 检查端口号配置，确认驱动程序已正确安装 |
| 动画生成失败 | 确保有足够磁盘空间，检查 ffmpeg 是否已安装 |

如遇其他问题，请参阅代码仓库中的详细文档或联系实验团队。

# 7 结论与心得

## 7.1 实验结论

本项目以无人机在复杂环境中的飞行控制问题为研究背景，将其抽象为经典的 FrozenLake 强化学习任务，构建了一个从理论建模、算法仿真到实机验证的全栈实验平台。通过系统性的实验设计与分析，得出以下核心结论：

**第一，强化学习算法能够有效解决离散状态空间下的路径规划问题**。在确定性环境中，Q-Learning、SARSA 等表格型算法以及 DQN 系列深度强化学习算法均能在标准 4×4 地图上达到 100% 的成功率，学习到的策略与理论最优路径完全一致。这一结果验证了强化学习方法在无人机导航任务中的可行性与有效性。

**第二，环境随机性对算法性能具有显著影响，不同算法类型呈现差异化的鲁棒性表现**。在随机性环境（is_slippery=True）中，深度强化学习算法展现出明显优势：Dueling DQN 达到 94% 成功率，DQN 与 PER-DQN 达到 92%，显著优于表格型算法的 70%–80%。规划类算法（A*、值迭代、策略迭代）在随机环境中几乎完全失效（成功率低于 5%），揭示了确定性规划方法在不确定环境中的根本局限。

**第三，算法选择需要权衡计算效率与性能表现**。表格型算法训练速度极快（通常在 0.5 秒以内），适合状态空间有限的场景；深度强化学习算法虽然计算开销较高（PER-DQN 需要 55.9 秒），但在复杂随机环境中的鲁棒性优势足以弥补这一代价。实验结果为不同应用场景下的算法选择提供了量化依据。

**第四，从仿真到实机的策略迁移具有实际可行性**。本项目成功将训练策略部署至真实无人机平台，在确定性与随机性两种模式下完成了实机飞行验证，实机成功率达到 85% 以上。这一结果表明，基于仿真环境训练的强化学习策略能够有效迁移至物理系统，为无人机自主控制的工程实践提供了可行路径。

## 7.2 实验心得与收获

### 7.2.1 对强化学习理论的深入理解

通过本次实验，我们对强化学习的核心概念有了更为深刻的认识。马尔可夫决策过程（MDP）的五元组建模方法为复杂控制问题提供了清晰的数学框架；贝尔曼方程揭示了状态价值与动作价值之间的递归关系，是理解所有强化学习算法的理论基石。在实现 16 种算法的过程中，我们深入理解了时序差分学习、蒙特卡洛方法与动态规划之间的联系与区别，以及同策略与异策略算法的本质差异。

特别是在对比确定性与随机性环境的实验中，我们直观地观察到了 Q 值分布从" 极化" 到" 扁平化" 的变化过程，深刻理解了环境不确定性如何影响信用分配与策略学习。随机性环境下 Q-Learning 需要 10 倍的训练回合数才能收敛，且最终成功率仍显著低于确定性环境，这一现象使我们认识到强化学习在真实物理环境中面临的挑战远超理想化的仿真设定。

### 7.2.2 工程实践能力的全面提升

本项目的实施涵盖了从算法实现、系统集成到实机部署的完整工程流程，极大地锻炼了团队的工程实践能力。

在软件工程方面，我们采用模块化设计思想构建了可扩展的实验平台，实现了算法、环境、实验配置与可视化的解耦。工厂模式的应用使得新算法的集成变得简洁高效；四级目录结构的结果存储体系确保了 800 组实验的有序管理与完全可复现性。这些工程实践经验对于未来从事大规模机器学习实验具有重要的参考价值。

在嵌入式系统开发方面，实机飞行测试使我们深入接触了串口通信协议、无线链路配置、实时控制系统等底层技术。调试过程中遇到的通信链路失效、指令同步错乱、信号断联等问题，迫使我们系统学习了嵌入式开发的调试方法与故障排查流程。这些经验表明，将算法从仿真环境迁移至物理系统并非简单的代码移植，而是需要充分考虑硬件约束、通信延迟、环境扰动等现实因素。

#### 7.2.3 团队协作与项目管理经验

本项目由五名成员共同完成，涉及算法实现、仿真平台开发、实机系统集成、可视化工具构建、文档撰写等多个并行任务。在项目推进过程中，我们建立了有效的分工协作机制：通过 Git 进行代码版本管理，通过结构化的配置文件实现实验参数的统一管理，通过自动化脚本降低重复劳动。

实验过程中也遇到了诸多挑战：算法调参的反复迭代、实机调试的意外故障、大规模实验的时间压力等。这些经历使我们认识到，科研项目的成功不仅依赖于技术能力，更需要合理的时间规划、清晰的任务分解与高效的团队沟通。

## 7.3 不足与展望

尽管本项目取得了预期成果，但仍存在若干不足之处，值得在后续研究中改进：

**算法层面**，当前实验主要聚焦于基于价值的强化学习方法，尚未涉及策略梯度算法（如 PPO、A2C）与 Actor-Critic 架构。这些算法在连续动作空间问题中具有独特优势，有望进一步提升无人机控制的精度与平滑性。此外，针对 n-step TD 与部分深度强化学习算法在特定场景下的性能不佳问题，可通过更系统的超参数搜索或自适应调参机制加以改进。

**环境建模层面**，当前 FrozenLake 环境对无人机飞行场景的抽象较为简化，未能充分反映真实飞行中的连续状态空间、复杂动力学模型与多维感知输入。后续研究可考虑引入更高保真度的无人机仿真环境（如 AirSim、Gazebo），以缩小仿真与实机之间的差距。

**实机验证层面**，当前实机测试主要在受控室内环境中进行，尚未涉及室外复杂场景。未来可扩展至户外环境，引入 GPS 定位、视觉感知等多源信息融合，验证强化学习策略在更真实条件下的泛化能力。

## 7.4 总结

本项目围绕" 深度强化学习在无人机飞行控制中的应用" 这一主题，完成了从理论学习、算法实现、仿真验证到实机部署的完整研究闭环。通过实现四大类共 16 种强化学习算法，在 24 张不同难度地图、确定性与随机性两种环境模式下进行了系统性的性能评估，最终成功将训练策略迁移至真实无人机平台。

实验结果不仅验证了强化学习方法在无人机路径规划任务中的有效性，更揭示了不同算法类型在效率、鲁棒性、可扩展性等维度上的差异化特征，为实际应用中的算法选择提供了量化参考。本项目的完成标志着团队在强化学习理论理解、工程实践能力与科研协作经验等方面均取得了显著提升，为后续深入研究奠定了坚实基础。